

09 — Schema Design: Logistics & Supply Chain

Table of Contents

- 1. [Learning Objectives](#)
- 2. [Domain Overview](#)
- 3. [ASCII ER Diagram](#)
- 4. [Complete DDL Script](#)
- 5. [Key Design Decisions](#)
- 6. [Common Queries](#)
- 7. [Geospatial Considerations](#)
- 8. [Common Mistakes](#)
- 9. [Best Practices](#)
- 10. [Performance Considerations](#)
- 11. [Interview Questions & Answers](#)
- 12. [Exercises with Solutions](#)
- 13. [Cross-References](#)

Learning Objectives

After completing this section you will be able to:

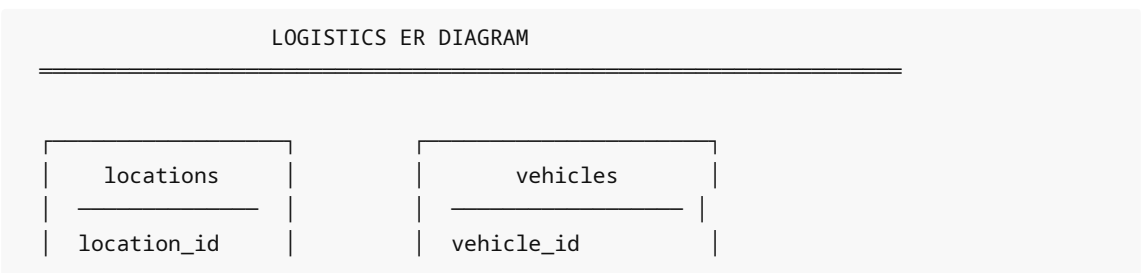
- Design a complete logistics schema covering shipments, routes, and tracking
- Model inventory across multiple warehouses
- Track shipment status history with an event-sourced approach
- Handle time-window constraints for deliveries
- Design for geospatial queries with POINT/PostGIS

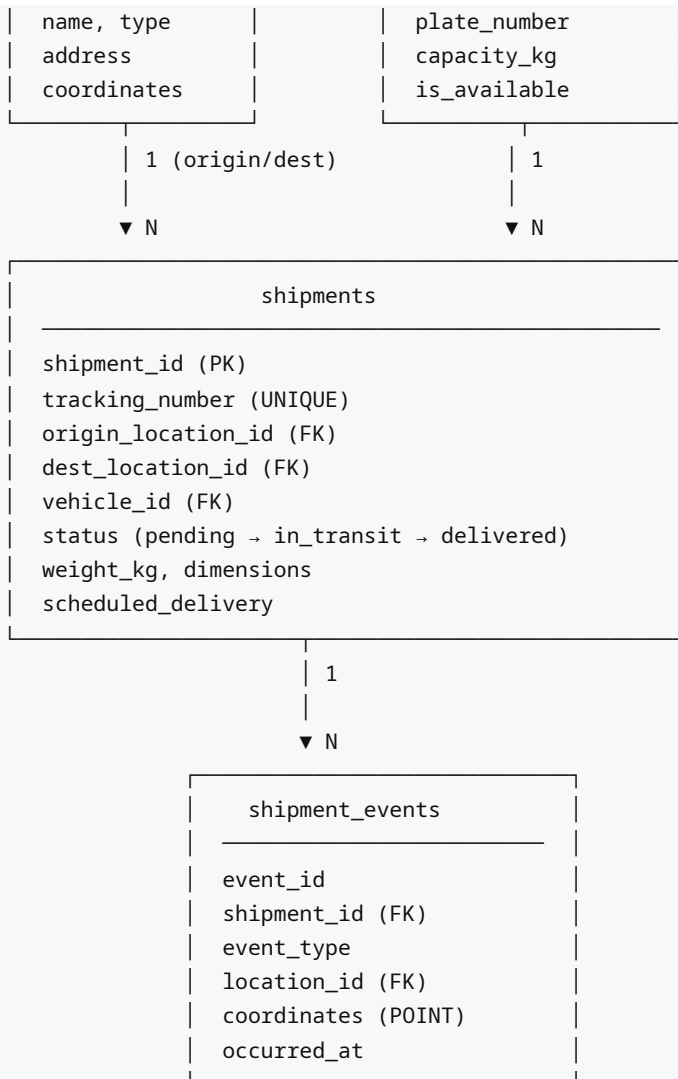
Domain Overview

A logistics platform manages:

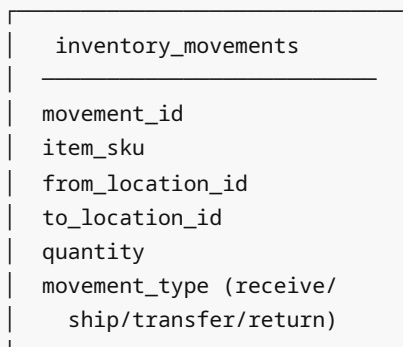
- **Locations:** warehouses, distribution centers, delivery addresses
- **Inventory:** stock levels, movements between locations
- **Shipments:** packages from origin to destination
- **Tracking Events:** real-time location and status updates
- **Routes:** planned paths through multiple stops
- **Vehicles & Drivers:** capacity and assignment tracking
- **Delivery Windows:** scheduled time slots for delivery

ASCII ER Diagram





Inventory Flow



Complete DDL Script

```

-- =====
-- LOGISTICS & SUPPLY CHAIN SCHEMA
-- =====

-- =====
-- LOCATIONS
-- =====

CREATE TABLE locations (
    location_id      INTEGER          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name             TEXT              NOT NULL,
    location_type     TEXT              NOT NULL
                                CHECK (location_type IN ('warehouse','distribution_center',
                                                         'pickup_point','delivery_address',
                                                         'port','airport','customs')),

    address_line1    TEXT,
    address_line2    TEXT,
    city             TEXT              NOT NULL,
    state_province   TEXT,
    postal_code       VARCHAR(20),
    country_code      CHAR(2)          NOT NULL,
    -- Geospatial coordinates (latitude, longitude)
    -- Use POINT for simple storage; PostGIS for advanced spatial queries
    latitude          DOUBLE PRECISION CHECK (latitude BETWEEN -90 AND 90),
    longitude         DOUBLE PRECISION CHECK (longitude BETWEEN -180 AND 180),
    timezone          TEXT              NOT NULL DEFAULT 'UTC',
    contact_phone     TEXT,
    operating_hours   JSONB,           -- {"mon":"08:00-18:00","sat":"09:00-14:00"}
    is_active         BOOLEAN          NOT NULL DEFAULT TRUE,
    created_at        TIMESTAMPTZ      NOT NULL DEFAULT now()
);

CREATE INDEX idx_locations_type ON locations (location_type);
CREATE INDEX idx_locations_country ON locations (country_code);
-- Spatial index (if using PostGIS):
-- CREATE INDEX idx_locations_geo ON locations USING GIST (
--     ST_MakePoint(longitude, latitude)::GEOGRAPHY
-- );

-- =====
-- VEHICLES & DRIVERS
-- =====

CREATE TABLE vehicles (
    vehicle_id        INTEGER          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    plate_number       VARCHAR(20)      NOT NULL UNIQUE,
    vehicle_type       TEXT              NOT NULL
                                CHECK (vehicle_type IN
('truck','van','motorcycle','plane','ship','train')),
    capacity_kg        NUMERIC(10,2)    NOT NULL CHECK (capacity_kg > 0),
    capacity_m3        NUMERIC(10,4),  -- cubic meters

```

```

        current_location_id INTEGER REFERENCES locations(location_id),
        is_available BOOLEAN NOT NULL DEFAULT TRUE,
        last_maintenance DATE,
        notes TEXT,
        created_at TIMESTAMPTZ NOT NULL DEFAULT now()
    );

CREATE TABLE drivers (
    driver_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    employee_id TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    license_number TEXT NOT NULL UNIQUE,
    license_expiry DATE NOT NULL,
    phone TEXT NOT NULL,
    current_vehicle_id INTEGER REFERENCES vehicles(vehicle_id),
    status TEXT NOT NULL DEFAULT 'available'
    CHECK (status IN
('available','on_route','off_duty','on_leave')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- =====
-- CUSTOMERS (senders and recipients)
-- =====

CREATE TABLE customers (
    customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_type TEXT NOT NULL DEFAULT 'individual'
    CHECK (customer_type IN ('individual','business')),
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT,
    account_number TEXT NOT NULL UNIQUE,
    credit_limit NUMERIC(12,4) NOT NULL DEFAULT 0,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- =====
-- SHIPMENTS
-- =====

CREATE TABLE shipments (
    shipment_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tracking_number VARCHAR(30) NOT NULL UNIQUE,
    -- Parties
    sender_customer_id BIGINT NOT NULL REFERENCES customers(customer_id),
    recipient_name TEXT NOT NULL,
    recipient_phone TEXT,
    recipient_email TEXT,
    -- Locations
    origin_location_id INTEGER NOT NULL REFERENCES locations(location_id),

```

```

dest_location_id    INTEGER          NOT NULL REFERENCES locations(location_id),
-- Delivery details
delivery_address    TEXT             NOT NULL,
delivery_city       TEXT             NOT NULL,
delivery_country    CHAR(2)          NOT NULL,
delivery_latitude   DOUBLE PRECISION,
delivery_longitude  DOUBLE PRECISION,
-- Scheduling
scheduled_pickup    TIMESTAMPTZ,
scheduled_delivery  TIMESTAMPTZ,
delivery_window_start TIMESTAMPTZ,
delivery_window_end TIMESTAMPTZ,
-- Physical properties
weight_kg           NUMERIC(10,3)    NOT NULL CHECK (weight_kg > 0),
length_cm           NUMERIC(8,2),
width_cm            NUMERIC(8,2),
height_cm           NUMERIC(8,2),
declared_value      NUMERIC(12,4),
currency_code       CHAR(3)          NOT NULL DEFAULT 'USD',
-- Classification
service_type        TEXT             NOT NULL DEFAULT 'standard'
                    CHECK (service_type IN
('standard', 'express', 'overnight', 'economy')),
priority            SMALLINT          NOT NULL DEFAULT 3 CHECK (priority BETWEEN 1
AND 5),
is_fragile          BOOLEAN           NOT NULL DEFAULT FALSE,
requires_signature  BOOLEAN           NOT NULL DEFAULT FALSE,
contains_hazmat     BOOLEAN           NOT NULL DEFAULT FALSE,
-- Status
status              TEXT             NOT NULL DEFAULT 'pending'
                    CHECK (status IN
('pending', 'pickup_scheduled', 'picked_up',
'in_transit', 'out_for_delivery', 'delivered',
'delivery_attempted', 'returned', 'lost', 'cancelled')),
-- Assignment
vehicle_id          INTEGER           REFERENCES vehicles(vehicle_id),
driver_id           INTEGER           REFERENCES drivers(driver_id),
-- Financial
shipping_cost       NUMERIC(10,4)    NOT NULL DEFAULT 0,
insurance_amount    NUMERIC(10,4)    NOT NULL DEFAULT 0,
-- Timestamps
created_at          TIMESTAMPTZ      NOT NULL DEFAULT now(),
updated_at          TIMESTAMPTZ      NOT NULL DEFAULT now(),
CONSTRAINT valid_delivery_window CHECK (
    delivery_window_end IS NULL OR
    delivery_window_end > delivery_window_start
),
CONSTRAINT different_origin_dest CHECK (origin_location_id != dest_location_id)
);

```

```

CREATE INDEX idx_shipments_tracking      ON shipments (tracking_number);
CREATE INDEX idx_shipments_sender        ON shipments (sender_customer_id);
CREATE INDEX idx_shipments_origin        ON shipments (origin_location_id);
CREATE INDEX idx_shipments_dest          ON shipments (dest_location_id);
CREATE INDEX idx_shipments_status        ON shipments (status, created_at DESC);
CREATE INDEX idx_shipments_driver        ON shipments (driver_id) WHERE driver_id IS
NOT NULL;
CREATE INDEX idx_shipments_vehicle        ON shipments (vehicle_id) WHERE vehicle_id IS
NOT NULL;
CREATE INDEX idx_shipments_scheduled     ON shipments (scheduled_delivery)
WHERE status NOT IN ('delivered','returned','cancelled');

-- =====
-- SHIPMENT EVENTS (Event Sourcing Pattern)
-- =====

CREATE TABLE shipment_events (
    event_id          BIGINT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    shipment_id       BIGINT          NOT NULL REFERENCES shipments(shipment_id),
    event_type        TEXT            NOT NULL
                                CHECK (event_type IN
('created','picked_up','arrived_at_hub',
                                'departed_hub','out_for_delivery',
                                'delivery_attempted','delivered',
                                'returned_to_sender','exception',
                                'location_update','status_change')),

    -- Location at time of event
    location_id       INTEGER         REFERENCES locations(location_id),
    latitude          DOUBLE PRECISION,
    longitude         DOUBLE PRECISION,
    -- Who recorded this
    recorded_by       TEXT,           -- driver ID, system, API
    notes             TEXT,
    metadata          JSONB          NOT NULL DEFAULT '{}',
    occurred_at       TIMESTAMPTZ     NOT NULL DEFAULT now(),
    received_at       TIMESTAMPTZ     NOT NULL DEFAULT now() -- when system received
it
);

CREATE INDEX idx_events_shipment ON shipment_events (shipment_id, occurred_at DESC);
CREATE INDEX idx_events_type     ON shipment_events (event_type, occurred_at DESC);
CREATE INDEX idx_events_location ON shipment_events (location_id) WHERE location_id
IS NOT NULL;

-- Trigger: automatically update shipment status from events
CREATE OR REPLACE FUNCTION sync_shipment_status()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    UPDATE shipments
    SET status = CASE NEW.event_type
        WHEN 'picked_up'      THEN 'picked_up'
        WHEN 'arrived_at_hub' THEN 'in_transit'
    END;
END;

```

```

        WHEN 'out_for_delivery' THEN 'out_for_delivery'
        WHEN 'delivered' THEN 'delivered'
        WHEN 'delivery_attempted' THEN 'delivery_attempted'
        WHEN 'returned_to_sender' THEN 'returned'
        ELSE status -- no change for other event types
    END,
    updated_at = now()
WHERE shipment_id = NEW.shipment_id;
RETURN NEW;
END;
$$;

CREATE TRIGGER trg_sync_shipment_status
AFTER INSERT ON shipment_events
FOR EACH ROW EXECUTE FUNCTION sync_shipment_status();

-- =====
-- ROUTES
-- =====

CREATE TABLE routes (
    route_id          INTEGER          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    route_code        TEXT             NOT NULL UNIQUE,
    vehicle_id        INTEGER          NOT NULL REFERENCES vehicles(vehicle_id),
    driver_id         INTEGER          NOT NULL REFERENCES drivers(driver_id),
    status            TEXT             NOT NULL DEFAULT 'planned'
                        CHECK (status IN
('planned', 'active', 'completed', 'cancelled')),
    planned_date      DATE             NOT NULL,
    started_at        TIMESTAMPTZ,
    completed_at      TIMESTAMPTZ,
    total_distance_km NUMERIC(10,2),
    created_at        TIMESTAMPTZ      NOT NULL DEFAULT now()
);

CREATE TABLE route_stops (
    stop_id           INTEGER          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    route_id          INTEGER          NOT NULL REFERENCES routes(route_id) ON DELETE
CASCADE,
    location_id       INTEGER          NOT NULL REFERENCES locations(location_id),
    stop_order        SMALLINT         NOT NULL,
    shipment_id       BIGINT           REFERENCES shipments(shipment_id),
    stop_type         TEXT             NOT NULL CHECK (stop_type IN
('pickup', 'delivery', 'hub', 'depot')),
    eta               TIMESTAMPTZ,
    arrived_at        TIMESTAMPTZ,
    departed_at       TIMESTAMPTZ,
    UNIQUE (route_id, stop_order)
);

CREATE INDEX idx_route_stops_route ON route_stops (route_id, stop_order);

```

```

-- =====
-- INVENTORY
-- =====

CREATE TABLE inventory_items (
    item_id          INTEGER          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sku              VARCHAR(50)      NOT NULL UNIQUE,
    name             TEXT             NOT NULL,
    unit_of_measure  TEXT             NOT NULL DEFAULT 'UNIT',
    weight_kg        NUMERIC(10,4),
    hazmat_class     TEXT
);

CREATE TABLE inventory_stock (
    item_id          INTEGER          NOT NULL REFERENCES inventory_items(item_id),
    location_id      INTEGER          NOT NULL REFERENCES locations(location_id),
    quantity         INTEGER          NOT NULL DEFAULT 0 CHECK (quantity >= 0),
    reserved_qty     INTEGER          NOT NULL DEFAULT 0 CHECK (reserved_qty >= 0),
    min_threshold    INTEGER          NOT NULL DEFAULT 0,
    updated_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    PRIMARY KEY (item_id, location_id),
    CONSTRAINT reserved_le_quantity CHECK (reserved_qty <= quantity)
);

CREATE TABLE inventory_movements (
    movement_id      BIGINT           GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    item_id          INTEGER          NOT NULL REFERENCES inventory_items(item_id),
    from_location_id INTEGER          REFERENCES locations(location_id),
    to_location_id   INTEGER          REFERENCES locations(location_id),
    quantity         INTEGER          NOT NULL CHECK (quantity > 0),
    movement_type    TEXT             NOT NULL
                                CHECK (movement_type IN ('receive','ship','transfer',
                                                         'return','adjustment','damage')),
    reference_id     TEXT,             -- shipment_id, PO number, etc.
    notes            TEXT,
    moved_at         TIMESTAMPTZ      NOT NULL DEFAULT now(),
    recorded_by      TEXT,
    CONSTRAINT movement_needs_location CHECK (
        from_location_id IS NOT NULL OR to_location_id IS NOT NULL
    )
);

CREATE INDEX idx_inv_movements_item      ON inventory_movements (item_id, moved_at
DESC);
CREATE INDEX idx_inv_movements_from_loc ON inventory_movements (from_location_id);
CREATE INDEX idx_inv_movements_to_loc   ON inventory_movements (to_location_id);

```

Key Design Decisions

1. **Event sourcing for shipment tracking** — `shipment_events` is append-only. The current `status` in `shipments` is a denormalized cache derived from events. This provides complete history and

enables time-travel queries.

2. **Explicit delivery window columns** — `delivery_window_start` and `delivery_window_end` allow the EXCLUDE constraint pattern for preventing driver schedule conflicts.
 3. **Separate inventory_stock and inventory_movements** — `inventory_stock` is the current state; `inventory_movements` is the history. Both are needed.
 4. **DOUBLE PRECISION for coordinates** — Sufficient for GPS-level precision (~10cm accuracy at the equator).
-

Common Queries

```
-- Track a shipment
SELECT
    se.occurred_at,
    se.event_type,
    l.name AS location_name,
    l.city,
    se.notes
FROM shipment_events se
LEFT JOIN locations l ON l.location_id = se.location_id
WHERE se.shipment_id = (
    SELECT shipment_id FROM shipments WHERE tracking_number = 'TRACK123'
)
ORDER BY se.occurred_at;

-- Today's deliveries for a driver
SELECT
    s.tracking_number,
    s.recipient_name,
    s.delivery_address,
    s.delivery_city,
    s.delivery_window_start,
    s.delivery_window_end,
    s.requires_signature,
    s.is_fragile
FROM shipments s
WHERE s.driver_id = :driver_id
    AND s.status = 'out_for_delivery'
    AND s.scheduled_delivery::DATE = CURRENT_DATE
ORDER BY s.delivery_window_start;

-- Warehouse inventory below reorder threshold
SELECT
    l.name AS warehouse,
    ii.sku,
    ii.name AS item_name,
    ist.quantity,
    ist.reserved_qty,
```

```

        ist.quantity - ist.reserved_qty AS available,
        ist.min_threshold
FROM inventory_stock ist
JOIN locations l ON l.location_id = ist.location_id
JOIN inventory_items ii ON ii.item_id = ist.item_id
WHERE ist.quantity - ist.reserved_qty <= ist.min_threshold
      AND l.location_type = 'warehouse'
ORDER BY l.name, available;

-- Late shipments (past scheduled delivery, not delivered)
SELECT
    s.tracking_number,
    s.status,
    s.scheduled_delivery,
    EXTRACT(EPOCH FROM (now() - s.scheduled_delivery))/3600 AS hours_late,
    s.recipient_name,
    s.delivery_city
FROM shipments s
WHERE s.scheduled_delivery < now()
      AND s.status NOT IN ('delivered','returned','cancelled')
ORDER BY s.scheduled_delivery;

```

Geospatial Considerations

```

-- With PostGIS: distance-based queries
CREATE EXTENSION IF NOT EXISTS postgis;

-- Add geography column to locations
ALTER TABLE locations ADD COLUMN geo GEOGRAPHY(POINT, 4326);
UPDATE locations SET geo = ST_MakePoint(longitude, latitude)::GEOGRAPHY
WHERE longitude IS NOT NULL AND latitude IS NOT NULL;

CREATE INDEX idx_locations_geo ON locations USING GIST (geo);

-- Find warehouses within 50km of a delivery point
SELECT l.name, l.city,
       ST_Distance(l.geo, ST_MakePoint(-74.006, 40.7128)::GEOGRAPHY) / 1000 AS
distance_km
FROM locations l
WHERE l.location_type = 'warehouse'
      AND ST_DWithin(l.geo, ST_MakePoint(-74.006, 40.7128)::GEOGRAPHY, 50000) -- 50km
in meters
ORDER BY distance_km;

```

Common Mistakes

1. **Updating shipment status directly** without creating an event — Loses history.
2. **Not indexing the `tracking_number` column** — This is the most common lookup column; must be indexed.

3. **Storing coordinates as TEXT** — Use DOUBLE PRECISION columns or PostGIS GEOGRAPHY.
 4. **No delivery time window constraint** — Allows scheduling two deliveries at the same time to the same driver.
 5. **Single location per shipment** — Real shipments pass through multiple hubs; events capture this.
-

Best Practices

1. **Treat shipment_events as append-only** — Never update or delete events.
 2. **Use PostGIS** for any real geospatial distance/area queries.
 3. **Index on status with partial index** — Only index active (non-terminal) shipments.
 4. **Separate physical address from location record** — Delivery addresses are per-shipment; warehouse addresses are per-location.
 5. **Use tracking_number as the external identifier** — Never expose internal shipment_id .
-

Performance Considerations

```
-- Event sourcing: efficient recent event lookup
CREATE INDEX idx_events_shipment_recent ON shipment_events (shipment_id)
INCLUDE (event_type, occurred_at, location_id)
WHERE occurred_at > now() - INTERVAL '90 days';

-- Partition large event tables by month
CREATE TABLE shipment_events_2024_01
PARTITION OF shipment_events
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- Active routes index
CREATE INDEX idx_routes_active ON routes (driver_id, planned_date)
WHERE status IN ('planned', 'active');
```

Interview Questions & Answers

Q1: How do you track a shipment's history without losing any information?

A: Use an event-sourcing pattern: a shipment_events table that is strictly append-only. Each scan, status change, or location update creates a new event row. The current status on the shipments table is a denormalized cache — the authoritative record is the full event sequence.

Q2: How would you handle the scheduling of drivers to avoid conflicts?

A: Use an EXCLUSION constraint with range types: EXCLUDE USING GIST (driver_id WITH =, delivery_window WITH &&) . This prevents two overlapping deliveries being assigned to the same driver at the database level.

Q3: What is the difference between storing inventory as current state vs. movement events?

A: Both are needed. inventory_stock (current state) enables fast "how much do we have now?" queries. inventory_movements (events) enable "what happened to this stock?" questions and provide an audit trail. The current state is computed from movements but stored for performance.

Q4: Why is DOUBLE PRECISION appropriate for GPS coordinates?

A: GPS latitude/longitude needs ~~6-7 significant decimal digits for sub-meter accuracy~~ (0.0001 degree $\approx$ 11 meters). DOUBLE PRECISION provides ~15 significant digits — more than sufficient. REAL (6 digits) is too imprecise for navigation.

Q5: How do you efficiently query "all shipments not delivered within the last 30 days"?

A: Use a partial index that only indexes non-terminal shipments: `CREATE INDEX idx_active_shipments ON shipments (scheduled_delivery) WHERE status NOT IN ('delivered', 'returned', 'cancelled')`. This keeps the index small and fast.

Exercises with Solutions

Exercise 1

Write a query to calculate on-time delivery rate by service type for the last 30 days.

Solution:

```
SELECT
  service_type,
  count(*) AS total_delivered,
  count(*) FILTER (WHERE status = 'delivered'
    AND updated_at <= scheduled_delivery) AS on_time,
  ROUND(100.0 * count(*) FILTER (WHERE status = 'delivered'
    AND updated_at <= scheduled_delivery) / count(*), 2) AS on_time_pct
FROM shipments
WHERE status = 'delivered'
  AND created_at >= now() - INTERVAL '30 days'
GROUP BY service_type
ORDER BY on_time_pct DESC;
```

Exercise 2

Design a function that records a new shipment event and returns the updated shipment status.

Solution:

```
CREATE OR REPLACE FUNCTION record_shipment_event(
  p_tracking_number TEXT,
  p_event_type TEXT,
  p_location_id INTEGER DEFAULT NULL,
  p_notes TEXT DEFAULT NULL,
  p_metadata JSONB DEFAULT '{}'
)
RETURNS TEXT -- new status
LANGUAGE plpgsql AS $$
DECLARE
  v_shipment_id BIGINT;
  v_new_status TEXT;
BEGIN
```

```

SELECT shipment_id INTO v_shipment_id
FROM shipments WHERE tracking_number = p_tracking_number;

IF NOT FOUND THEN
    RAISE EXCEPTION 'Shipment not found: %', p_tracking_number;
END IF;

INSERT INTO shipment_events (shipment_id, event_type, location_id, notes,
metadata)
VALUES (v_shipment_id, p_event_type, p_location_id, p_notes, p_metadata);

SELECT status INTO v_new_status FROM shipments WHERE shipment_id =
v_shipment_id;
RETURN v_new_status;
END;
$$;

SELECT record_shipment_event('TRACK123', 'out_for_delivery', 5, 'On truck for
delivery');

```

Cross-References

- [05_schema_design_ecommerce.md](#) — order fulfillment connects to logistics
- [06_schema_design_banking.md](#) — payment for shipping
- [10_design_patterns.md](#) — event sourcing, audit trail
- [../05_PostgreSQL_Core/07_special_types.md](#) — range types for delivery windows
- [../07_Indexes/](#) — spatial indexes with PostGIS